



Amlogic

NN 工具 FAQ

Revision: 1.0

Release Date: 2024-07-17

Copyright

© 2024 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 , and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

变更记录

版本 1.0 (2024-07-17)

第一版。

Confidential!
nick@khadas.com
2024-11-20 08:49:14

目录

变更记录.....	ii
1. 框架.....	1
1.1 当前 Amlogic NN 支持哪几种开源框架?	1
1.2 当前 Amlogic NN 支持什么格式的模型?	1
1.3 当前 Amlogic NN 支持哪些通用模型?	1
1.4 NPU 上是否能同时运行多个 nn 任务	1
2. 工具.....	2
2.1 Adla_tool 工具量化方式是否与 tensorflow 量化方式一致?	2
2.2 工具运行时出错: xxx.so No such file or directory	2
3. 模型转换	3
3.1 模型转换会有精度损失吗?	3
3.2 量化时候, dataset 需要多少张图片? 数量和精度是否成正比?	3
3.3 使用 adla_tool 工具导出 case 代码时, 如何设置对应板子正确的参数?	3
3.4 caffe 模型转换过程中报错: Not supported caffe net model version(v0 layer or v1 layer)	4
3.5 量化输入数据只能是图片么? 模型输入数据不是图片的怎么给量化数据?	4
3.6 Caffe 模型常见问题.....	4
4. 集成类	6
4.1 模型转换过程中出错, 如何确认问题点?	6
4.2 板子上运行结果精度差, 如何定位?	8
4.3 demo 集成后, 前处理时间较长, 是整个流程中的计算瓶颈, 怎么解决?	9
4.4 demo 集成后, 发现后处理过程中的反量化时间很长, 怎么解决?	10
4.5 如何得到每个 op 的运算结果和耗时?	11
4.6 如何确认 NPU 的利用率	11
4.7 如何获取模型运行的带宽?	12

1. 框架

1.1 当前 Amlogic NN 支持哪几种开源框架？

解答：当前支持的神经网络框架以及对应版本信息如下：

Framework	Version	Model load(Remark)
Tensorflow	2.15.0	Tf.compat.v1.lite.TFLiteConverter.from_frozen_graph()
Pytorch	2.0.0	torch.jit.load(model_path)
Onnx	1.12.0	onnx.load(model_path)
Mxnet	1.6.0	mx.model.load_checkpoint(prefix_path, prefix_epoch)
Darknet	https://github.com/sjusa/samuel/darknet	
Caffe	apt-get install caffe-cpu	caffe.Net(proto_file,blob_file,caffe.TEST)
tflite	2.15.0	tflite.Model.GetRootAsModel tflite.Model.Model.GetRootAsModel
Keras	3.0.5	tf.keras.models.load_model(model_path)
Paddle	2.4.2	Paddle.jit.load(model_path)

1.2 当前 Amlogic NN 支持什么格式模型？

解答：当前 Amlogic NN 只支持量化后的模型，需将支持的开源框架对应的模型进行量化处理。 量化方式有三种 Int8、int16、Uint8，支持 perlayer 和 perchannel 量化。

1.3 当前 Amlogic NN 支持哪些通用模型？

解答：SqueezeNet、VGG、Resnet*、Inception、RetinaNet*、 MobileNet*、 LSTM*、GRU*、SSD、Yolo*、FasterRCNN、transformer* 。建议直接使用 adla_tool 进行模型转换，能正常生成 adla 文件的模型都支持。

1.4 NPU 上是否能同时运行多个 nn 任务

解答：支持多线程多进程操作，但底层是单进程单帧方式处理的。

2. 工具

2.1 Adla_tool 工具量化方式是否与 tensorflow 量化方式一致？

解答：是一致的。当前支持 int8/int16/uint8 三种量化方式

2.2 工具运行时出错：xxx.so No such file or directory

```
adla-toolkit-binary-3.0.7.4/bin$ ./adla_convert  
[13979] Error loading Python lib '/mnt/e/adla-toolkit-binary-  
3.0.7.4/bin/libpython3.10.so.1.0': dlopen: /mnt/e/adla-toolkit-binary-  
3.0.7.4/bin/libpython3.10.so.1.0: cannot open shared object file: No such file or directory
```

解答：工具需要在 linux 环境下解压直接后使用。解压后的文件夹，需避免再进行文件夹方式的拷贝移动。

3. 模型转换

3.1 模型转换会有精度损失吗？

解答：模型转换涉及到 float 数据量化成 int8/uint8/int16 数据，会有一定程度上的精度损失。Int16 perchannel 一般情况下最接近原始 float 精度。量化精度排序: int8 perlayer < int8 perchannel < int16 perchannel

3.2 量化时候，dataset 需要多少张图片？数量和精度是否成正比？

解答：一般建议图片数量在 200~2000 张左右，尽量是模型运行时场景的图片，或者是训练模型验证集里的图片。这对量化效果有正向作用。但数量和精度不成正比关系。

建议：使用 ./bin/adla_plug_in/tools/Quantize_optimization_tool 来搜索最优量化参数。

3.3 使用 adla_tool 工具导出 case 代码时，如何设置对应板子正确的参数？

解答：执行命令：cat /proc/cpuinfo 或者 cat /proc/cpu_chipid

查看倒第二行数 Serial 对应的数列码前四个字符(例如 3d0a 等)，根据如下对应关系来确定设置的值：

平台	PID	Serial
C3	0XA001	Serial: 3d0a010100000000c613492931563343
S5	0XA002	Serial: 3e0a0202000000007c404b6c31563553
T7C	0XA003	Serial: 360c01030000000025f904ab31563541
T3X	0XA004	Serial: 420a010100000000daff6e4931583354
S6	0XA005	Serial: 480a0201000000006827632931563653

例如：

序列号前四位为 3d0a 时，设置参数 “--target-platform PRODUCT_PID0XA001”；

序列号前四位为 3e0a 时，设置参数 “--target-platform PRODUCT_PID0XA002”；

序列号前四位为 360c 时，设置参数 “--target-platform PRODUCT_PID0XA003”；

序列号前四位为 420a 时，设置参数 “--target-platform PRODUCT_PID0XA004”；

序列号前四位为 480a 时，设置参数 “--target-platform PRODUCT_PID0XA005”；

如果序列号前四位在上述表格中没有统计出来，当前建议使用序列号前三位的对应的参数尝试下。

3.4 caffe 模型转换过程中报错：Not supported caffe net model version(v0 layer or v1 layer)

解答：当前 caffemodel 或者 prototxt 文件版本过老，需要使用工具更新下 prototxt 或者 caffe 模型文件

解决方案：更新 prototxt 工具:upgrade_net_proto_text-d。

更新 caffemodel 工具:upgrade_net_proto_binary-d

获取方式：编译 caffe 源码，tools 目录下会生成对应的可执行文件

3.5 量化输入数据只能是图片么？模型输入数据不是图片的怎么给量化数据？

解答：量化数据可以不是图片。当前可以将数组或者 txt 格式文件数据保存成 npy 文件，然后量化的时候，dataset.txt 里面使用 npy 文件的路径替换图片路径。如：

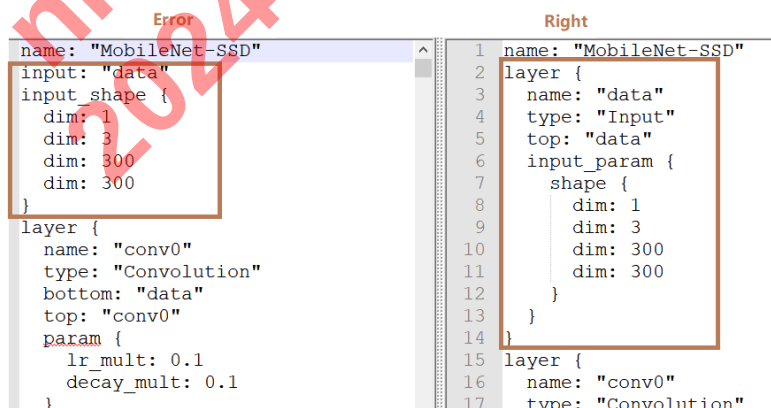
使用或者参照 acuity_tools_xxx/
bin/adla_plug_in/tools/input_npy_txt_bin_generate.py 脚本，保存 npy 文件。

```
python3 input_npy_txt_bin_generate.py 1,3,224,224 npy np.float32 0 255
```

3.6 Caffe 模型常见问题

解答：

a、caffe 模型 input 层的格式：



b、prototxt 中出现不支持 layer 时容易导致转换失败，例如检测类模型，不支持 DetectionOutput layer，需要手动删掉该 layer。


```
0 layer {  
1   name: "detection_out"  
2   type: "DetectionOutput"  
3   bottom: "mbox_loc"  
4   bottom: "mbox_conf_flatten"  
5   bottom: "mbox_priorbox"  
6   top: "detection_out"  
7   include {  
8     phase: TEST  
9   }  
0   detection_output_param {  
1     num_classes: 21  
2     share_location: true  
3     background_label_id: 0  
4     nms_param {  
5       nms_threshold: 0.45  
6       top_k: 100  
7     }  
8   }  
9 }
```

Confidential!
nick@khadas.com
2024-11-20 08:49:14

4. 集成类

4.1 模型转换过程中出错，如何确认问题点？

解答：

模型转换失败，可以先设置环境变量: `export ADLA_LOG_LEVEL=DEBUG`，再进行一次模型转换。

(1) 确认错误出现在哪一步

在转换 log 中搜索关键字: `Adla_tool`，当前转换过程中，搜索关键字出现的结果有下面几种：

I Adla_tool: step_0 enter, begin to load original model.

I Adla_tool: step_0 out, load original model successfully.

I Adla_tool: step_1 enter, frontend begin to parse model.

I Adla_tool: step_1 out, frontend parse model successfully.

I Adla_tool: step_2 enter, begin to convert model to ir.

I Adla_tool: step_2 out, convert model to ir successfully.

I Adla_tool: step_3 enter, begin to quantize model.

I Adla_tool: step_3 out, quantize model successfully.

I Adla_tool: step_4 enter, begin to convert adla_file.

I Adla_tool: step_4 out, convert adla_file successfully.

通过确认该流程 log，能初步确认当前是在哪一步处理中出现异常

(2) 不同位置出错的处理方法

Step_0: 模型加载错误:

如果模型搜索关键字之后，只出现了 `step_0 enter` 的 log，那表明原始模型加载失败，此时需要客户根据 1.1 章节表格中各个 framework 对应的版本以及模型加载方式确认原始模型是否存在问题。

```

I Namespace(batch_size=1, channel_mean_value=None, default_ranges_max=None, default_ranges_min=None,
input_shapes='3,640,640', inputs='input', iterations=1, mean=0, model='yolox.pt', model_type='pytorch',
e_source_file=None, std_dev=1, target_platform='PROPIETARY_PID0XA001', weights=None)
Adla_tool: step_0 enter, begin to load original model.
E
Unknown type name 'NoneType':
Serialized File "code/_torch_/torchvision/extension.py", line 1
def _assert_has_ops() -> NoneType:
~~~~~ <--- HERE
    _0 = _torch_.torchvision.extension._has_ops
    _1 = "Couldn't load custom C++ ops. This can happen if your PyTorch and torchvision versions are in
on on the compatible versions, check https://github.com/pytorch/vision#installation for the compatibility
on with torchvision.__version__ and verify if they are compatible, and if not please reinstall torchvisi
'_assert_has_ops' is being compiled since it was called from 'nms'
Serialized File "code/_torch_/torchvision/ops/boxes.py", line 22
    scores: Tensor,
    iou_threshold: float) -> Tensor:
~~~~~ <--- HERE
    _6 = _torch_.torchvision.extension._assert_has_ops
    _7 = _6()
    _8 = ops.torchvision.nms(boxes, scores, iou_threshold)
'nms' is being compiled since it was called from '_batched_nms_coordinate_trick'
File "/home/dengliu/anaconda3/envs/python3.8/lib/python3.8/site-packages/torchvision/ops/boxes.py",
offsets = idxs.to(boxes) * (max_coordinate + torch.tensor(1).to(boxes))
boxes_for_nms = boxes + offsets[:, None]
keep = nms(boxes_for_nms, scores, iou_threshold)
~~~~ <--- HERE
return keep
Serialized File "code/_torch_/torchvision/ops/boxes.py", line 16
    _5 = torch.unsqueeze(torch.slice(offsets), 1)
    boxes_for_nms = torch.add(boxes, _5)
    keep = _torch_.torchvision.ops.boxes.nms(boxes_for_nms, scores, iou_threshold, )
~~~~~ <--- HERE
    _0 = keep
return _0
'_batched_nms_coordinate_trick' is being compiled since it was called from 'YOLOX.forward'
Serialized File "code/_torch_/mmdet/models/detectors/yolox.py", line 11
def forward(self: _torch_.mmdet.models.detectors.yolox.YOLOX,
img: Tensor) -> Tuple[Tensor, Tensor]:
    _0 = _torch_.torchvision.ops.boxes._batched_nms_coordinate_trick
~~~~~ <--- HERE
    _1 = _torch_.torchvision.ops.boxes._batched_nms_coordinate_trick
    _2 = _torch_.torchvision.ops.boxes._batched_nms_coordinate_trick
I -----warning(0)-----

```

Step_1: 前端解析模型出错:

需要将模型提供给 Amlogic 进行 debug.

Step_2: 转换 IR 时出错:

可先和 Amlogic 确认下 log 提示信息是否明确, 如果 log 信息不足以定位出问题原因, 需考虑将模型提供给 Amlogic 进行 debug.

Step_3: 模型量化转换过程中出错:

确认下异常 log 是出现 kill 还是其他异常, 如:

```

Adla_tool: step_3 enter, begin to quantize model.
I Quantize info: Input type: <dtype: 'int8'>, quant_type: int8, output_type: <dtype: 'int8'>, disable_per_channel: True, quantizer: True
I Quantize info: enable hybrid quantization. Generate hybrid quantization model
2024-07-12 11:26:54.808022: W tensorflow/compiler/mlir/python/tf_tfl_flatbuffer_helpers.cc:378] Ignored output_format.
2024-07-12 11:26:54.808057: W tensorflow/compiler/mlir/python/tf_tfl_flatbuffer_helpers.cc:381] Ignored drop_control_dependency.
2024-07-12 11:26:54.814064: I tensorflow/cc/saved_model/loader.cc:83] Reading SavedModel from: Qwen-0_5B_dynamic_adla_unify/Qwen-0_5B_dynamic
2024-07-12 11:26:55.420124: I tensorflow/cc/saved_model/loader.cc:51] Reading meta graph with tags { serve }
2024-07-12 11:26:55.420157: I tensorflow/cc/saved_model/loader.cc:146] Reading SavedModel debug info (if present) from: Qwen-0_5B_dynamic_adla_unify/Qwen-0_5B_dynamic
2024-07-12 11:26:56.198603: I tensorflow/compiler/mlir/mlir_graph_optimization_pass.cc:388] MLIR V1 optimization pass is not enabled
2024-07-12 11:26:56.254081: I tensorflow/cc/saved_model/loader.cc:233] Restoring SavedModel bundle.
2024-07-12 11:27:03.798600: I tensorflow/cc/saved_model/loader.cc:217] Running initialization op on SavedModel bundle at path: Qwen-0_5B_dynamic_adla_unify/Qwen-0_5
Killed

```

如果是出现 kill, 可能是服务器运行内存不足导致, 建议在模型转换过程中, 使用 free -g 看下剩余内存:

```

dengliu@amlogic-Precision-3650-Tower:~$ free -g
              total        used        free      shared  buff/cache   available
Mem:           125          1          16           0         107        122
Swap:          127          0         127

```

解决方案：参考 <https://blog.csdn.net/A308114/article/details/118544964> 增大交换内存(Swap)或者更换服务器。

Note：增大交换内存后，会占用系统盘空间，客户可以在模型转换后，再将交换空间改回原大小。

Setp_4：编译器转换出错：

可先和 Amlogic 确认下 log 提示信息是否明确。如果 log 信息不足定位出问题，可以将模型转换中间层导出成 quantized tflite 发给 amlogic 进行 debug。

导出方法：模型转换前设置环境变量： `export ADLA_EXPORT_MIDDLE_TO_TFLITE=True`，再进行模型转换。

4.2 板子上运行结果精度差，如何定位？

解答：

(1) 确认 float 结果与原始模型精度是否一致

当前工具支持将模型转换过程中的 IR 导出成 float tflite 模型，先确认导出的 float tflite 结果与原始模型结果是否保持一致。

导出方式：

- 设置环境变量 `export ADLA_EXPORT_MIDDLE_TO_TFLITE=True`
- 模型转换时，量化类型选择 float32，如：`--quantize-dtype float32`
- 此时输出目录下会保存出 float tflite 模型，可以跟原始模型进行结果比对
- 注意如果原始模型是 nchw 格式，注意在测试 float tflite 时，使用相同的前处理，然后再多做一次 transpose 即可。

(2) 确认 float 结果和 quantized 的结果是否保持一致

将模型转换过程中的 IR 导出 float tflite 和 quantized tflite，确认 float 结果与 quantized 的结果是否保持一致。

Quantized 导出方式：

- 设置环境变量 `export ADLA_EXPORT_MIDDLE_TO_TFLITE=True`
- 正常进行模型量化转换，保存量化文件文件，生成 quantized tflite

(3) 确认 quantized 的结果与板端是否保持一致

将 pc 侧的 quantized tflite 结果与板侧结果进行对比，注意保持输入一致性。板侧输出的数据格式跟 tflite 一样，是 NHWC 格式。可以通过 NNIDE_Tool 里的 `demo_adla_compare_buf.py` 来进行 quantized tflite 和板侧结果对比。

问题分析：

- 如果第一步结果异常：

确认输入一致性后，如果结果不一致，说明 adla_tool 在模型转换过程中适配错误。需将模型以及量化图片、量化参数信息提供给 Amlogic 进行 debug。

b. 如果第二步结果异常：

第一步结果 ok，第二步结果异常，说明是量化精度问题，需要做以下确认：

- 首先先 check 下量化参数 channel-mean-value 等是否正确，量化数据集 dataset.txt 中是否提供足够的图片或者 npy 文件，一般建议 200~500 张。
- 三种量化方式精度排序：int8 perlayer < int8 perchannel < int16 perchannel，如果量化输入数据 dataset 正常，channel-mean-value 也正常，可以尝试使用 int8 perchannel 或者 int16 perchannel 量化
- 确认哪种量化方式精度满足需求，如果都存在精度问题，需将模型以及量化图片、量化参数信息提供给 Amlogic 进行 debug。

c. 如果第三步结果异常：

第二步结果正常，第三步结果出错，说明是底层 driver 或者 compiler 的错误。可以做如下尝试：

- 转换模型之前，设置环境变量：export ADLA_PRECISE_MODEL=True，生成新的 adla 文件，再进行板端测试。
- 如果结果正常，说明是 compiler 转换融合过程中出错导致，客户可以先使用该方式规避下精度问题。如果结果仍旧错误，说明 driver 中存在 bug，可以将生成的 quantized tflite 发给 amlogic 进行 debug。

4.3 demo 集成后，前处理时间较长，是整个流程中的计算瓶颈，怎么解决？

解答：demo 前处理步骤：

- a. 对原始数据 data0 做预处理，得到 data1(如分类模型，读图后做归一化处理)
- b. 对预处理后的数据 data1 做量化，得到量化数据 data2
- c. 将量化数据 data2 送给 NPU 进行推理

方案一：尝试将前处理步骤中的第一二步融合

以分类模型为例：

原始数据 data0：为图片数据，range(0~255)。

预处理后的数据 data1：对 data0 做归一化处理，data1=data0/255,range(0~1)

或者 data1=(data0-128)/128,range(-1~1)

量化数据 data2：对 data1 做量化，data2=data/scale-zero_point。

因为前处理的操作，和 scale/zero_point 的数值都是已知的，可以直接将公式展开，将常量代入进去，直接通过 data0 计算出 data2，节省前处理和量化时间。有的模型

代入后, $\text{data2}=(\text{uint8})\text{data0}$ 或者 $\text{data2}=(\text{int8})(\text{data0}-128)$ 。其他非图片类的模型, 也可以参考该方法确认能否节省前处理和量化的时间。

对应的 scale 和 zero_point 在转 adla 后, 可以通过 adlainfo 获取:

`/xxxx/bin/adla_plug_in/tools/adla_info_tool/adlainfo_release -io xxxx.adla`

如下图, resnetnet 的 input scale: 0.007782, zero_point: 1

```
adla file name : caffe_output/resnet-18_int8.adla
*****
Inputs                                     : [0 ]
  Input[0]                                : 0
  Dim Count                               : 4
  Size of dim[0]                          : 1
  Size of dim[1]                          : 224
  Size of dim[2]                          : 224
  Size of dim[3]                          : 3
  type                                    : Int8
  scale                                   : [0.007782 ]
  zero_point                              : [1 ]
  inputs_memory_size                      : 147.00 kb
*****
Outputs                                    : [96 ]
  Output[0]                               : 96
  Dim Count                               : 2
  Size of dim[0]                          : 1
  Size of dim[1]                          : 1000
  type                                    : Int8
  scale                                   : [0.003906 ]
  zero_point                              : [-128 ]
  outputs_memory_size                     : 0.98 kb
*****
config:
  max_outstanding_outputs                 : 4
  axi_sram_size                           : 262144 bytes
*****
```

方案二: 提高第三步的效率

一般是调用 `aml_module_input_set` 接口给将量化数据 `data2` 送给 NPU 进行推理。当前可以参考 ADLA_SDK_API 文档, 调用 `aml_util_mallocBuffer` 接口申请 input/output buffer, 并将输入/输出格式设置为: `INPUT_DMA_DATA`。在做前处理时, 直接将处理数据写到通过 `aml_util_mallocBuffer` 接口申请到的 buffer 中即可。这样可以节省 data 从 DDR 拷贝到 NPU 的时间。

4.4 demo 集成后, 发现后处理过程中的反量化时间很长, 怎么解决?

解答:

NPU 只支持定点数据, 做后处理时, 使用的是 float 数据。一般在获取到 NPU 的结果后, 需要先对定点数据进行反量化操作, 转成 float, 再做后处理。但如果输出 buffer 较大, 反量化操作耗时较多。

当前建议方案: 直接基于定点数据先做处理, 再进行选择性的进行反量化操作。

	可能出现的现象	Note
cat utilization	please [echo 1 > utilization] first	没有打开监控开关，请先执行 1.1 步骤
	adla utilization : -1%	当前 adla 没有执行计算，可能是任务队列无任务，也可能是处于任务调度状态
	adla utilization : xx%	xx 为非 -1 值，此时获取的 xx 即为 adla 实时利用率

4.7 如何获取模型运行的带宽？

解答：

使用 adla_tool 进行模型转换后，会生成 case demo。当前只需要在 case demo 中添加 aml_util_enableProfile()和 aml_util_getProfileInfo()的调用即可。可参考 ADLA_SDK_API 文档 3.5 章节,或者参考代码如下：

invoke_time: npu 耗时

BW(Ddr_data): 带宽占用

memory_usage_bytes: 内存消耗

```
context = aml_module_create(&config);
aml_profile_config_t profile_data;
aml_util_enableProfile(context, &profile_data);
ret = run_network(netType, context, jpath);
aml_util_getProfileInfo(context, &profile_data);

fprintf(stdout, "\nProfiling data:\n");
fprintf(stdout, "model_name: %s\n", config.path);

fprintf(stdout, "-----from system-----\n");
fprintf(stdout, "core_freq_cur : %llu Hz\n", profile_data.profiling_data.ext.core_freq_cur);
fprintf(stdout, "axi_freq_cur : %llu Hz\n", profile_data.profiling_data.ext.axi_freq_cur);
fprintf(stdout, "mem_pool_size : 0x%08llX Bytes\n", profile_data.profiling_data.ext.mem_pool_size);
fprintf(stdout, "mem_pool_used : 0x%08llX Bytes\n", profile_data.profiling_data.ext.mem_pool_used);
fprintf(stdout, "mem_allocated_base : 0x%08llX Bytes\n", profile_data.profiling_data.ext.mem_allocated_base);
fprintf(stdout, "mem_allocated_umd : 0x%08llX Bytes\n", profile_data.profiling_data.ext.mem_allocated_umd);
int elapsed_hw_sw = (int)profile_data.profiling_data.ext.us_elapsed_in_hw_op + (int)profile_data.profiling_data.ext.us_elapsed_in_sw_op;

fprintf(stdout, "-----aml need-----\n");
fprintf(stdout, " : %7.2f ms, inference_time \n", (float)profile_data.profiling_data.inference_time_us / 1000);
fprintf(stdout, " : %7.2f ms, elapsed_in_hw_op \n", (float)profile_data.profiling_data.ext.us_elapsed_in_hw_op / 1000);
fprintf(stdout, " : %7.2f ms, elapsed_in_sw_op \n", (float)profile_data.profiling_data.ext.us_elapsed_in_sw_op / 1000);
fprintf(stdout, " : %7.2f ms, elapsed(hw_op+sw_op) \n", (float)elapsed_hw_sw / 1000);
fprintf(stdout, " : %7.2f ms, invoke_time \n", (float)usVal_inference / 1000);
fprintf(stdout, " : %7.2f FPS(pse) \n", (double)1000000 / (double)profile_data.inference_time_us);
fprintf(stdout, " : %7.2f FPS(hw+sw) \n", (double)1000000 / (double)elapsed_hw_sw);
fprintf(stdout, " : %7.2f FPS(invoke) \n", (double)1000000 / (double)(usVal_inference));
fprintf(stdout, " : %7.2f Mbytes, dram_read_size \n", (float)profile_data.profiling_data.dram_read_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes, dram_write_size \n", (float)profile_data.profiling_data.dram_write_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes, BW(Ddr_data) \n", (double)(profile_data.profiling_data.dram_read_bytes + profile_data.profiling_data.dram_write_bytes) / (1024 * 1024));

fprintf(stdout, " : %7.2f Mbytes, sram_read_bytes \n", (float)profile_data.profiling_data.sram_read_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes, sram_write_bytes \n", (float)profile_data.profiling_data.sram_write_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes, BW(Sram_data) \n", (double)(profile_data.profiling_data.sram_read_bytes + profile_data.profiling_data.sram_write_bytes) / (1024 * 1024));

fprintf(stdout, " : %7.2f Mbytes, memory_usage_bytes \n", (float)profile_data.profiling_data.memory_usage_bytes / (1024 * 1024));

aml_util_disableProfile(context, &profile_data);
```